PYTHON ACTIVITY-BASED LEARNING

Assignment: Solutions for Any 10 Case Studies

Name	M. Naga sai sri
Branch	CSM
Section	2
Roll Number	26000949

Course: Python Programming

Prepared from the supplied Python Activity-Based Learning case-study sheet.

Contents

1. Progressive Electricity Billing

2. Hospital Billing and Patient Classification

3. Employee Payroll and Deduction Analysis

4. Retail Billing with Discount Rules

5. Examination Result and Grace-Mark Validation

6. Attendance Recovery Computation

7. ATM Transaction Sequence Simulation

8. Warehouse Inventory Reconciliation

9. Student Marks Matrix Analysis

10. Password Policy Validation Engine

Note: The sample outputs are text captures of representative runs. Execute each source-code block in Python to reproduce the output.

1. Progressive Electricity Billing

Problem Statement

Calculate an electricity bill from monthly units using progressive consumption slabs, a fixed charge, and a rebate for low usage.

Objectives

Use numeric variables, arithmetic, conditionals, and formatted output to calculate a transparent bill.

Assumptions

First 100 units cost ■1.50/unit; units 101–200 cost ■2.50/unit; units above 200 cost ■4.00/unit. Fixed charge is ■50. A 5% rebate applies when consumption is at most 100 units. Negative units are invalid.

Algorithm

1. Read units and validate that units are not negative.

2. Calculate energy charge slab by slab.

3. Add fixed charge; apply the low-usage rebate if eligible.

4. Display the energy charge, rebate, and final bill.

Flowchart

```
[Start] → [Input units] → {Units ≥ 0?}\n   No → [Show invalid input] → [End]\n   Yes → [Calculate slab charge
```

Pseudocode

```
BEGIN\n  INPUT units\n  IF units < 0: PRINT invalid\n  ELSE:\n    IF units <= 100: energy = units * 1.50\n    ELSE IF units <
```

Python Source Code

```
units = float(input("Enter units consumed: "))
if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 1.50
    elif units <= 200:
        energy = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
    subtotal = energy + 50
    rebate = subtotal * 0.05 if units <= 100 else 0
    total = subtotal - rebate
    print(f"Energy charge: ■{energy:.2f}")
    print(f"Fixed charge: ■50.00")
    print(f"Rebate: ■{rebate:.2f}")
    print(f"Final bill: ■{total:.2f}")
```

Test Cases

Units	Expected result
80	Energy ■120.00; rebate ■8.50; bill ■161.50
150	Energy ■275.00; rebate ■0; bill ■325.00
250	Energy ■550.00; rebate ■0; bill ■600.00
-5	Invalid units

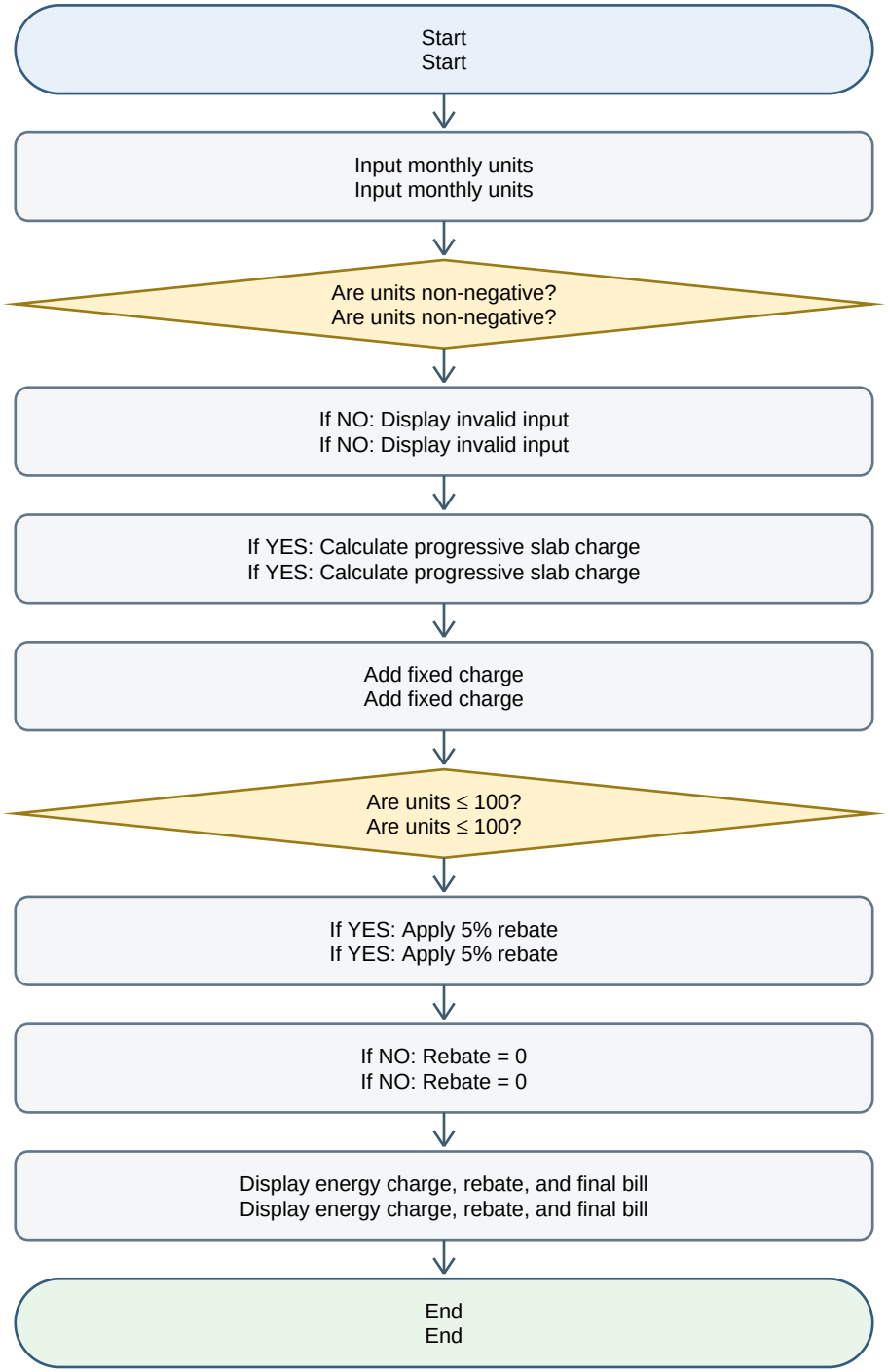
Sample Output

```
Enter units consumed: 80
Energy charge: ■120.00
Fixed charge: ■50.00
Rebate: ■8.50
Final bill: ■161.50
```

Conclusion

1. Progressive Electricity Billing

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



2. Hospital Billing and Patient Classification

Problem Statement

Calculate a patient's total hospital bill from consultation, diagnostic, and treatment charges, then apply a concession when the patient is a senior citizen.

Objectives

Practise input, validation, arithmetic, and conditional statements.

Assumptions

All charges must be non-negative. Senior citizens aged 60 or above receive a 10% concession on the subtotal. No other taxes or fees are included.

Algorithm

1. Read age and the three charges.

2. Reject negative age or charges.

3. Add charges to get the subtotal.

4. Apply a 10% concession for age 60 or above and print the amount due.

Flowchart

```
[Start] → [Input age and charges] → {Inputs valid?}\n No → [Invalid input] → [End]\n Yes → [Add charges] → {A
```

Pseudocode

```
BEGIN\n INPUT age, consultation, diagnostics, treatment\n IF age < 0 OR any charge < 0: PRINT invalid\n ELSE:\n   subtotal =
```

Python Source Code

```
age = int(input("Patient age: "))
consultation = float(input("Consultation charge: "))
diagnostics = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))

if age < 0 or min(consultation, diagnostics, treatment) < 0:
    print("Invalid input: age and charges must be non-negative")
else:
    subtotal = consultation + diagnostics + treatment
    discount = subtotal * 0.10 if age >= 60 else 0
    payable = subtotal - discount
    print(f"Subtotal: ■{subtotal:.2f}")
    print(f"Concession: ■{discount:.2f}")
    print(f"Amount payable: ■{payable:.2f}")
    print("Classification:", "Senior patient" if age >= 60 else "Regular patient")
```

Test Cases

Input	Expected result
Age 65; charges 500, 300, 1200	Subtotal ■2000; concession ■200; payable ■1800
Age 30; charges 500, 300, 1200	Subtotal ■2000; concession ■0; payable ■2000
Age 50; one charge -10	Invalid input

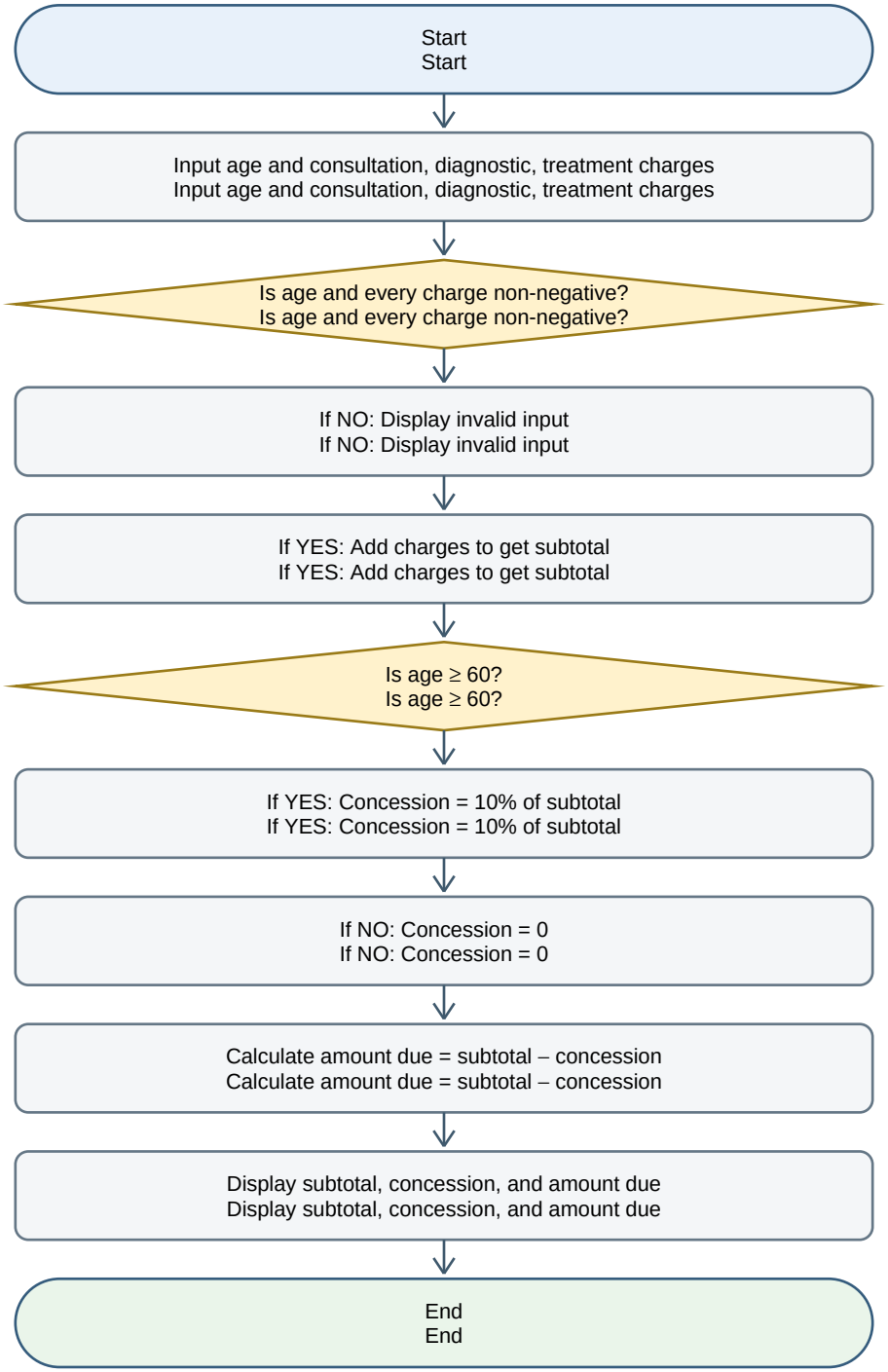
Sample Output

```
Patient age: 65
Consultation charge: 500
Diagnostic charge: 300
Treatment charge: 1200
Subtotal: ■2000.00
Concession: ■200.00
Amount payable: ■1800.00
Classification: Senior patient
```

Conclusion

2. Hospital Billing and Patient Classification

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



3. Employee Payroll and Deduction Analysis

Problem Statement

Prepare a salary statement from basic pay, allowances, overtime, and a fixed deduction rate.

Objectives

Use variables, operators, conditional validation, and formatted numeric output.

Assumptions

HRA is 20% of basic pay, other allowance is 10%, overtime is paid at ■150 per hour, and deduction is 8% of gross salary. Inputs cannot be negative.

Algorithm

1. Read basic pay and overtime hours.

2. Validate both inputs.

3. Calculate HRA, allowance, overtime pay, and gross salary.

4. Calculate deductions and net salary; print the salary statement.

Flowchart

```
[Start] → [Read pay and overtime] → {Non-negative?}\n No → [Invalid] → [End]\n Yes → [Calculate allowances +
```

Pseudocode

```
BEGIN\n INPUT basic, overtime_hours\n IF basic < 0 OR overtime_hours < 0: PRINT invalid\n ELSE:\n   hra = basic * 0.20; allow
```

Python Source Code

```
basic = float(input("Basic pay (■): "))
hours = float(input("Overtime hours: "))

if basic < 0 or hours < 0:
    print("Invalid input")
else:
    hra = basic * 0.20
    allowance = basic * 0.10
    overtime = hours * 150
    gross = basic + hra + allowance + overtime
    deduction = gross * 0.08
    net = gross - deduction
    print("\n--- SALARY STATEMENT ---")
    print(f"Basic pay: ■{basic:.2f}")
    print(f"HRA: ■{hra:.2f}")
    print(f"Other allowance: ■{allowance:.2f}")
    print(f"Overtime pay: ■{overtime:.2f}")
    print(f"Gross salary: ■{gross:.2f}")
    print(f"Deduction: ■{deduction:.2f}")
    print(f"Net salary: ■{net:.2f}")
```

Test Cases

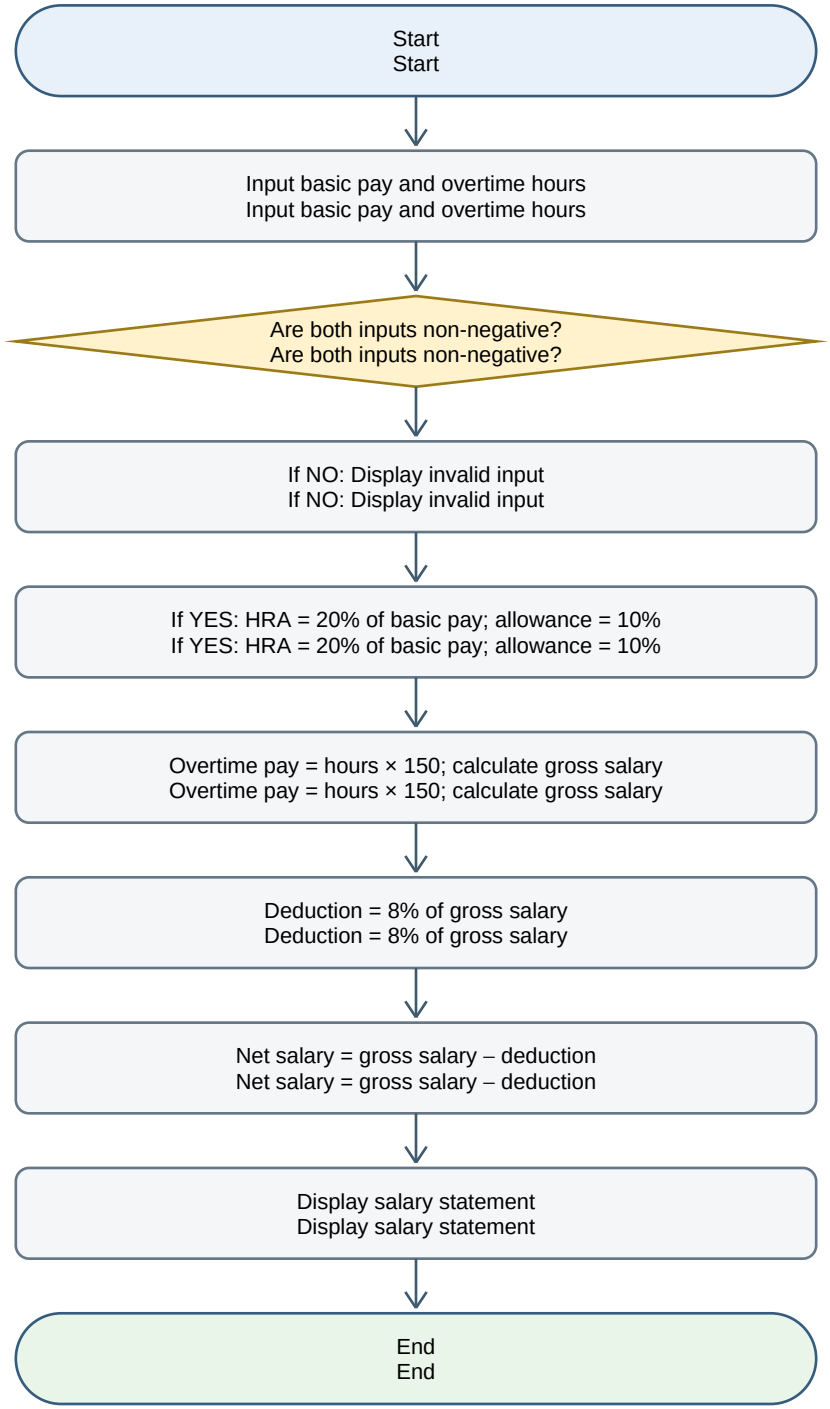
Input	Expected result
Basic 20000; overtime 5 hours	Gross ■27,750; deduction ■2,220; net ■25,530
Basic 15000; overtime 0	Gross ■19,500; deduction ■1,560; net ■17,940
Basic -1; overtime 2	Invalid input

Sample Output

```
Basic pay (■): 20000
Overtime hours: 5
--- SALARY STATEMENT ---
Basic pay: ■20000.00
HRA: ■4000.00
Other allowance: ■2000.00
Overtime pay: ■750.00
Gross salary: ■26750.00
```

3. Employee Payroll and Deduction Analysis

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



4. Retail Billing with Discount Rules

Problem Statement

Calculate a retail bill for multiple products using quantities, a threshold discount, and tax.

Objectives

Practise loops, lists, arithmetic, and if/else logic.

Assumptions

Enter the price and quantity for each item. Prices must be non-negative and quantities must be positive. A 10% discount applies when the pre-tax subtotal is at least ■2,000. GST is modelled as 5% of the discounted amount.

Algorithm

1. Read the number of products and validate it.

2. For each product, read price and quantity and accumulate line totals.

3. Apply the threshold discount to the subtotal.

4. Calculate tax on the discounted amount and display the invoice.

Flowchart

```
[Start] → [Input item count] → [Loop through items; add price × quantity]\n → [Subtotal] → {Subtotal ≥ ■2000?
```

Pseudocode

```
BEGIN\n INPUT n\n subtotal = 0\n REPEAT n times:\n   INPUT price, quantity\n   IF price < 0 OR quantity <= 0: report invalid
```

Python Source Code

```
n = int(input("Number of products: "))
subtotal = 0.0
valid = n > 0

if not valid:
    print("Enter at least one product")
else:
    for i in range(1, n + 1):
        price = float(input(f"Product {i} price: "))
        quantity = int(input(f"Product {i} quantity: "))
        if price < 0 or quantity <= 0:
            valid = False
            break
        subtotal += price * quantity

    if not valid:
        print("Invalid product price or quantity")
    else:
        discount = subtotal * 0.10 if subtotal >= 2000 else 0
        taxable = subtotal - discount
        tax = taxable * 0.05
        print(f"Subtotal: ■{subtotal:.2f}")
        print(f"Discount: ■{discount:.2f}")
        print(f"GST (5%): ■{tax:.2f}")
        print(f"Amount payable: ■{taxable + tax:.2f}")
```

Test Cases

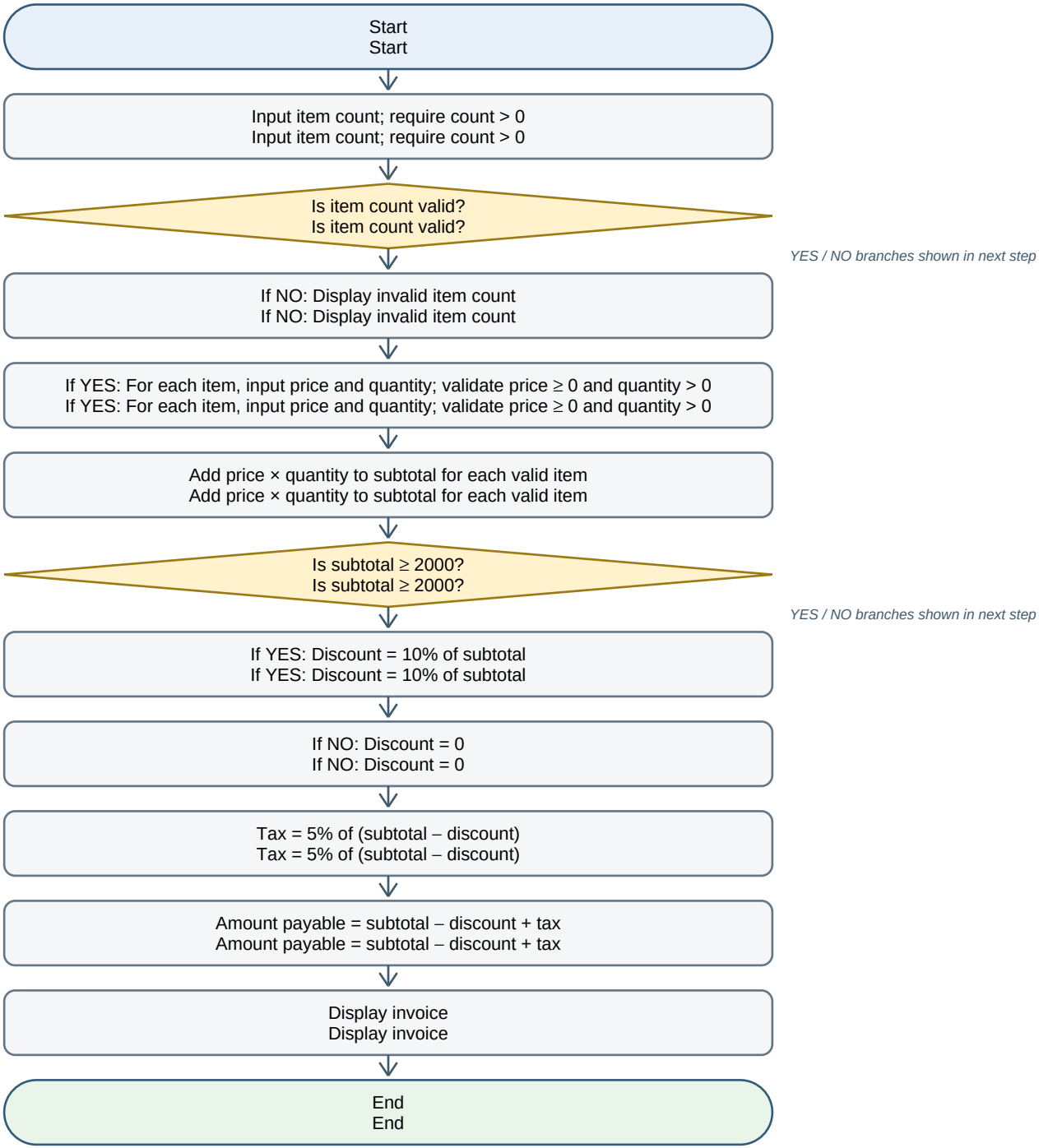
Input	Expected result
Items ■1200×1, ■1000×1	Subtotal ■2200; discount ■220; GST ■99; total ■2079
Item ■500×2	Subtotal ■1000; discount ■0; GST ■50; total ■1050
Quantity 0	Invalid product price or quantity

Sample Output

```
Number of products: 2
Product 1 price: 1200
Product 1 quantity: 1
```

4. Retail Billing with Discount Rules

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



5. Examination Result and Grace-Mark Validation

Problem Statement

Determine pass/fail status from marks in several subjects, with a limited grace-mark rule.

Objectives

Use lists, loops, conditionals, and validation of maximum marks.

Assumptions

There are three subjects, each out of 100. Pass mark is 35 in each subject. If exactly one subject has 33 or 34, grace marks may raise it to 35, but the student must have at least 35 in the other two subjects. Grace marks cannot exceed 2.

Algorithm

1. Read three subject marks and validate each is between 0 and 100.

2. Find subjects below 35.

3. If exactly one subject is 33 or 34 and the others pass, add the required grace marks.

4. Print final marks and result.

Flowchart

```
[Start] → [Read 3 marks] → {All marks in 0..100?}\n No → [Invalid] → [End]\n Yes → [Find marks below 35] → {
```

Pseudocode

```
BEGIN\n INPUT marks list\n IF any mark < 0 OR mark > 100: PRINT invalid\n ELSE:\n   low = subjects with marks < 35\n   IF len
```

Python Source Code

```
marks = []
for subject in range(1, 4):
    mark = int(input(f"Marks for subject {subject} (0-100): "))
    marks.append(mark)

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks")
else:
    low = [i for i in range(3) if marks[i] < 35]
    grace_used = 0
    if len(low) == 1 and marks[low[0]] in (33, 34):
        other_marks_pass = all(marks[i] >= 35 for i in range(3) if i != low[0])
        if other_marks_pass:
            grace_used = 35 - marks[low[0]]
            marks[low[0]] = 35

    print("Final marks:", marks)
    print("Grace marks used:", grace_used)
    print("Result:", "PASS" if all(m >= 35 for m in marks) else "FAIL")
```

Test Cases

Marks	Expected result
34, 60, 70	Final [35, 60, 70]; grace 1; PASS
32, 60, 70	Final [32, 60, 70]; grace 0; FAIL
34, 34, 70	No grace; FAIL
101, 50, 60	Invalid marks

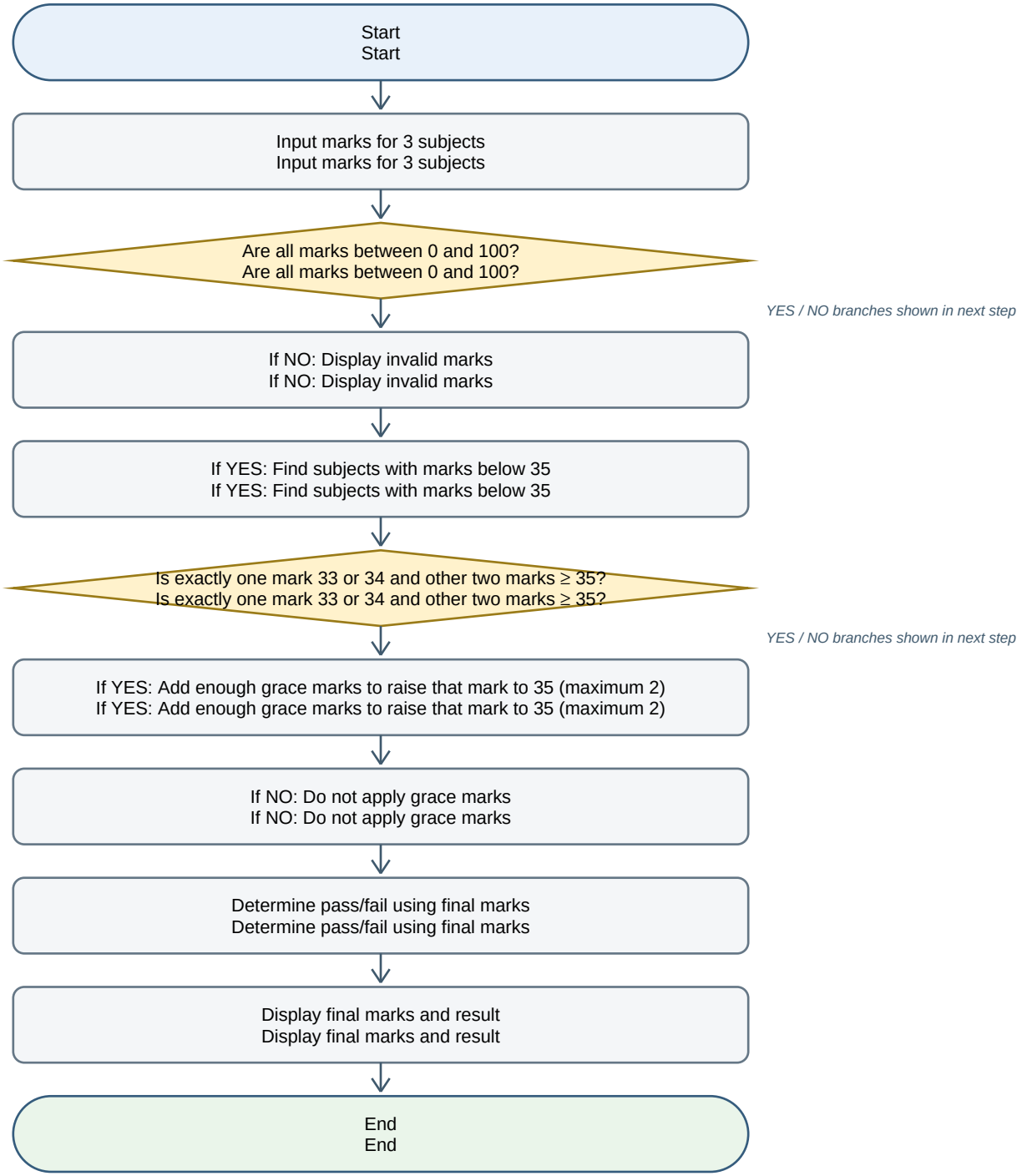
Sample Output

```
Marks for subject 1 (0-100): 34
Marks for subject 2 (0-100): 60
Marks for subject 3 (0-100): 70
Final marks: [35, 60, 70]
Grace marks used: 1
Result: PASS
```

Conclusion

5. Examination Result and Grace-Mark Validation

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



6. Attendance Recovery Computation

Problem Statement

Calculate current attendance and the minimum number of consecutive classes a student must attend to reach a target percentage.

Objectives

Apply formulas, loops, and conditional logic to solve a small iterative problem.

Assumptions

Classes attended cannot exceed classes held. The target is 75%. Future classes are all attended. If current attendance already meets the target, the required number is zero.

Algorithm

1. Read attended classes and total classes held.

2. Validate the counts.

3. Calculate the current percentage.

4. Increase a future-attended counter until the attendance ratio reaches 75%.

5. Print current percentage and required classes.

Flowchart

```
[Start] → [Input attended, held] → {Valid counts?}\n No → [Invalid] → [End]\n Yes → [Calculate percentage] →
```

Pseudocode

```
BEGIN\n INPUT attended, held\n IF held < 0 OR attended < 0 OR attended > held: PRINT invalid\n ELSE IF held == 0: PRINT no cl
```

Python Source Code

```
attended = int(input("Classes attended: "))
held = int(input("Total classes held: "))

if held < 0 or attended < 0 or attended > held:
    print("Invalid attendance counts")
elif held == 0:
    print("No classes have been held yet")
else:
    current = attended / held * 100
    future = 0
    while (attended + future) / (held + future) * 100 < 75:
        future += 1
    print(f"Current attendance: {current:.2f}%")
    print(f"Consecutive classes to attend: {future}")
```

Test Cases

Input	Expected result
60 attended, 100 held	Current 60.00%; attend 60 more classes
80 attended, 100 held	Current 80.00%; attend 0 more classes
10 attended, 0 held	No classes have been held yet
12 attended, 10 held	Invalid attendance counts

Sample Output

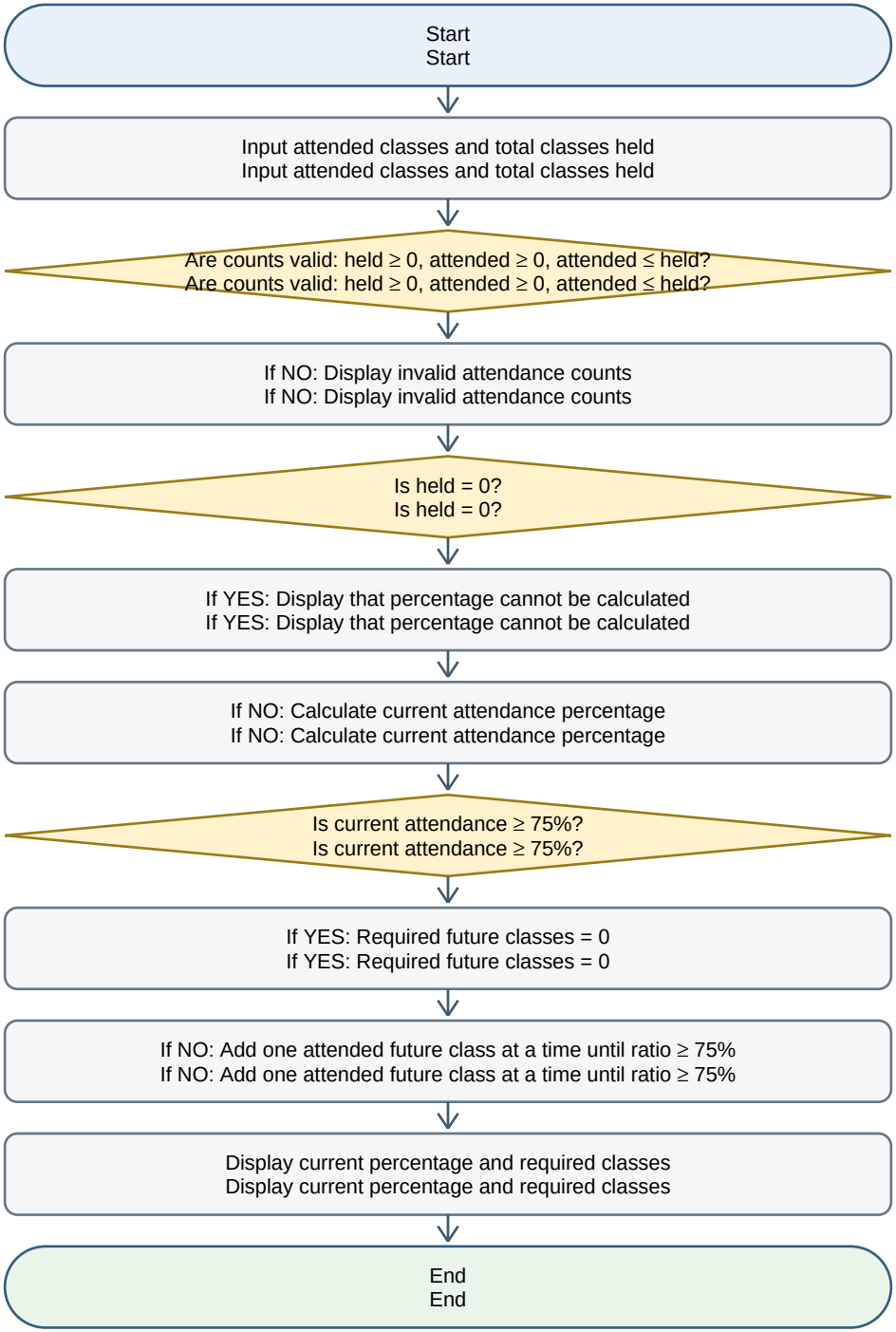
```
Classes attended: 60
Total classes held: 100
Current attendance: 60.00%
Consecutive classes to attend: 60
```

Conclusion

The loop finds the smallest future class count that meets the attendance target under the stated assumption.

6. Attendance Recovery Computation

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



7. ATM Transaction Sequence Simulation

Problem Statement

Simulate deposits and withdrawals, maintain a balance, and reject invalid transactions or withdrawals exceeding the available balance.

Objectives

Practise loops, lists, conditionals, and transaction summaries.

Assumptions

The starting balance is ■5,000. A transaction is represented by a type (D for deposit or W for withdrawal) and a positive amount. Withdrawals cannot exceed the current balance. Invalid transactions are rejected without changing the balance.

Algorithm

1. Initialize the balance and transaction summary.
2. Read the number of transactions.
3. For each transaction, validate type and amount.
4. Process valid deposits or withdrawals; reject insufficient-funds withdrawals.
5. Display final balance and accepted/rejected counts.

Flowchart

```
[Start] → [Balance = ■5000] → [Input transaction count]\n → [Read type and amount] → {Valid type and amount?}
```

Pseudocode

```
BEGIN\n balance = 5000; accepted = 0; rejected = 0\n INPUT n\n FOR each transaction:\n   INPUT type, amount\n   IF amount <=
```

Python Source Code

```
balance = 5000.0
accepted = 0
rejected = 0
n = int(input("Number of transactions: "))

for i in range(n):
    kind = input("Transaction type (D/W): ").strip().upper()
    amount = float(input("Amount (■): "))

    if amount <= 0 or kind not in ("D", "W"):
        print("Transaction rejected: invalid type or amount")
        rejected += 1
    elif kind == "D":
        balance += amount
        accepted += 1
        print("Deposit accepted")
    elif amount <= balance:
        balance -= amount
        accepted += 1
        print("Withdrawal accepted")
    else:
        print("Transaction rejected: insufficient balance")
        rejected += 1

print(f"Final balance: ■{balance:.2f}")
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

Test Cases

Transactions	Expected result
D 1000; W 500	Balance ■5500; accepted 2; rejected 0
W 6000	Balance ■5000; accepted 0; rejected 1
D -20	Balance ■5000; accepted 0; rejected 1

Sample Output

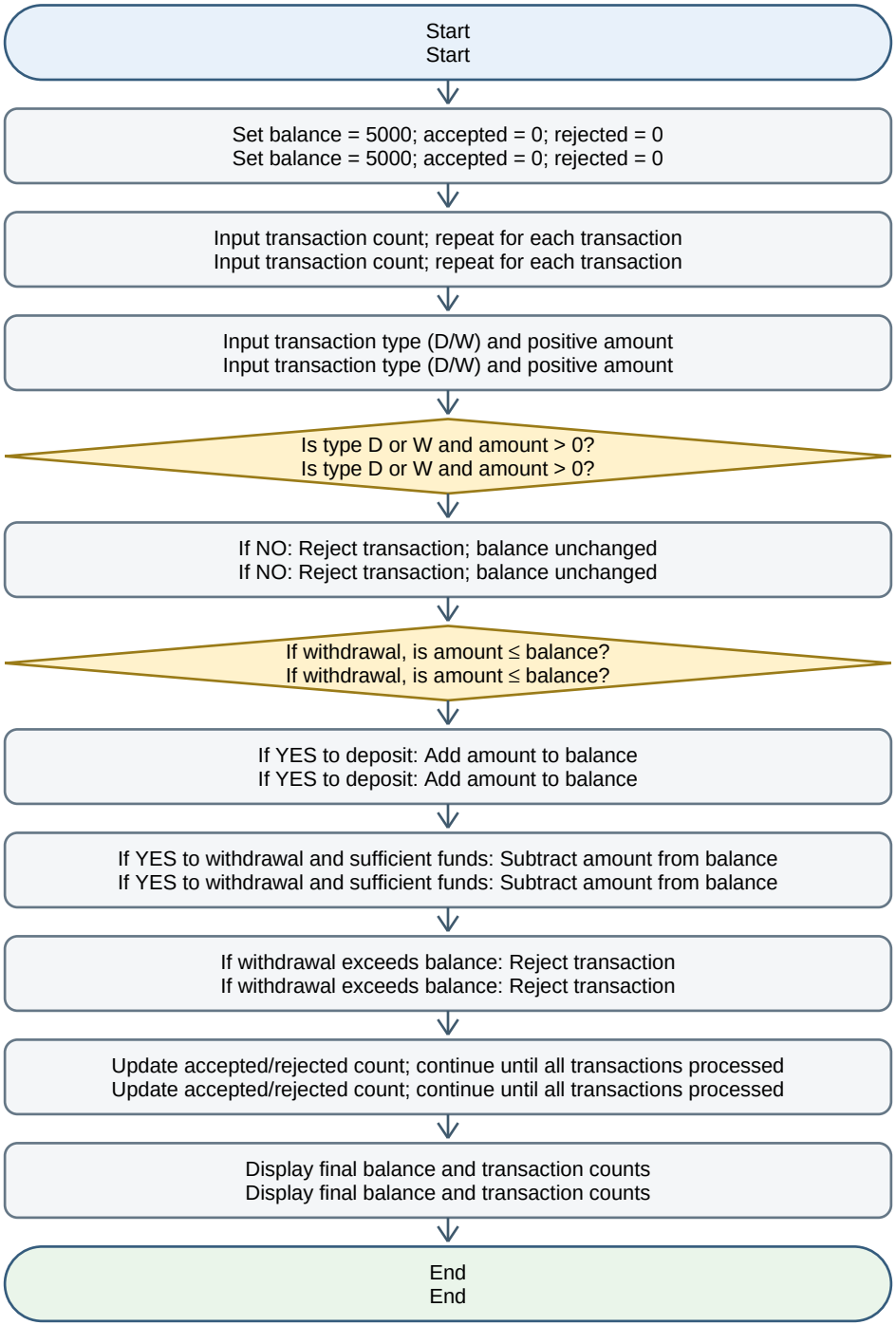
```
Number of transactions: 2
Transaction type (D/W): D
Amount (■): 1000
Deposit accepted
Transaction type (D/W): W
Amount (■): 500
Withdrawal accepted
Final balance: ■5500.00
Accepted transactions: 2
Rejected transactions: 0
```

Conclusion

The simulation updates balances only for valid transactions and reports a clear transaction summary.

7. ATM Transaction Sequence Simulation

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



8. Warehouse Inventory Reconciliation

Problem Statement

Compare opening stock, received stock, dispatched stock, and recorded closing stock to find discrepancies and replenishment needs.

Objectives

Use lists, loops, arithmetic, and conditional statements for inventory checks.

Assumptions

Quantities are non-negative integers. Expected closing stock = opening + received – dispatched. Replenishment is the amount needed to reach a target stock of 20 units if the actual closing stock is below 20.

Algorithm

1. Read product names and four stock quantities for each product.
2. Calculate expected closing stock.
3. Compare expected closing stock with recorded closing stock.
4. Calculate discrepancy and replenishment to reach 20 units.
5. Print a report for every product.

Flowchart

```
[Start] → [Input product records] → [Calculate expected closing]\n → {Expected = recorded?} Yes → [No discrepa
```

Pseudocode

```
BEGIN\n INPUT n\n FOR each product:\n   INPUT name, opening, received, dispatched, recorded\n   expected = opening + received
```

Python Source Code

```
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    received = int(input("Received quantity: "))
    dispatched = int(input("Dispatched quantity: "))
    recorded = int(input("Recorded closing stock: "))

    if min(opening, received, dispatched, recorded) < 0:
        print("Invalid quantities; record skipped")
        continue

    expected = opening + received - dispatched
    discrepancy = recorded - expected
    replenishment = max(0, 20 - recorded)
    status = "OK" if discrepancy == 0 else "CHECK DISCREPANCY"
    print(f"{name}: expected={expected}, recorded={recorded}")
    print(f"Difference={discrepancy}, replenishment={replenishment}, status={status}")
```

Test Cases

Record	Expected result
Pen: opening 30, received 10, dispatched 25, recorded 15	Expected 15; difference 0; replenishment 5; OK
Book: opening 10, received 5, dispatched 4, recorded 12	Expected 11; difference 1; replenishment 8; discrepancy flagged
Any quantity -1	Record skipped as invalid

Sample Output

```
Product name: Pen
Opening stock: 30
```

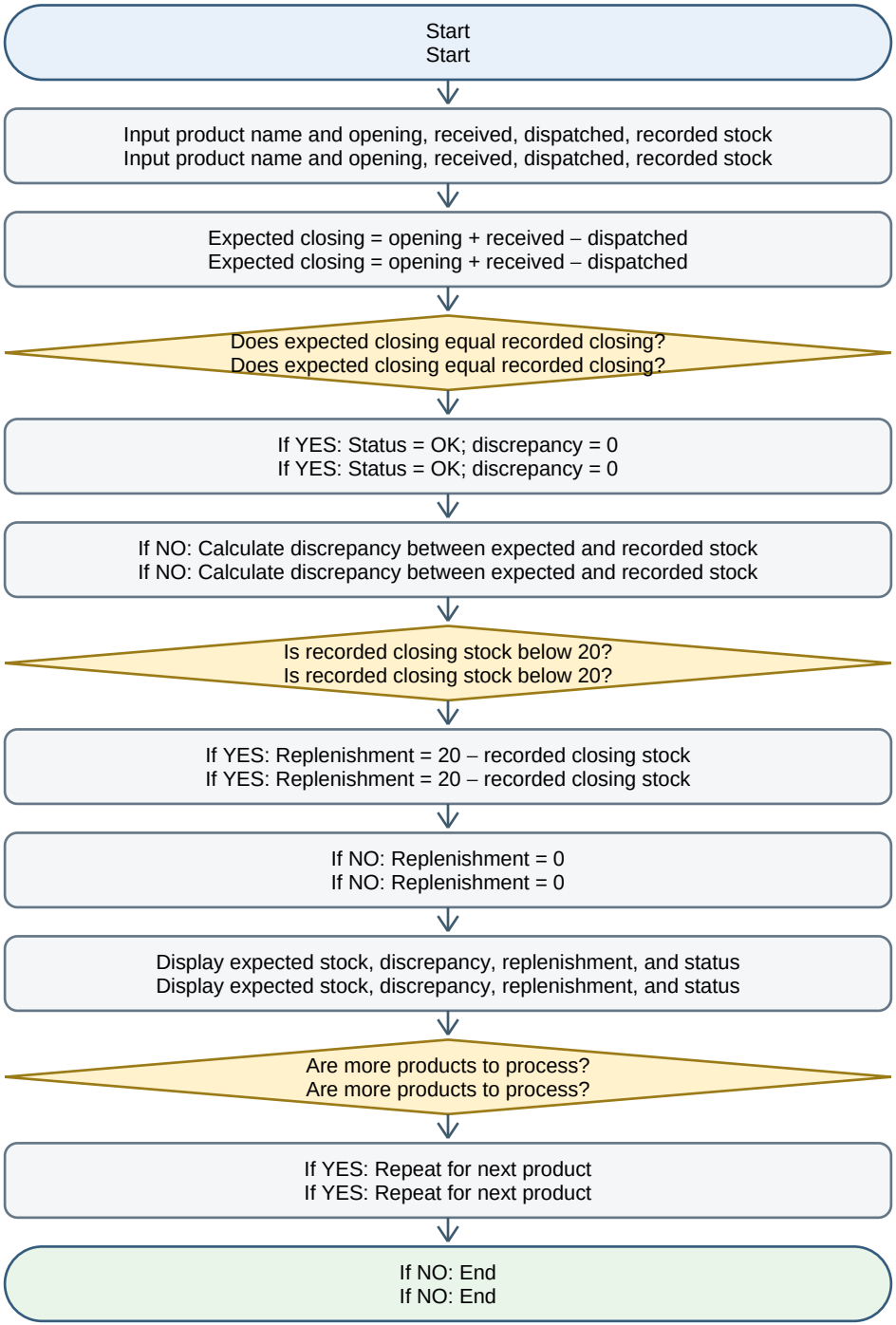
```
Received quantity: 10
Dispatched quantity: 25
Recorded closing stock: 15
Pen: expected=15, recorded=15
Difference=0, replenishment=5, status=OK
```

Conclusion

The reconciliation logic identifies mismatches and computes replenishment against the chosen target stock.

8. Warehouse Inventory Reconciliation

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



9. Student Marks Matrix Analysis

Problem Statement

Analyse marks stored in a nested list to calculate each student's total and average and identify the top performer.

Objectives

Practise nested lists, loops, row-wise calculations, and comparisons.

Assumptions

There are three students and three subjects. Each mark must be between 0 and 100. If students tie on total marks, the first student in the input order is reported as the top performer.

Algorithm

1. Read a 3×3 matrix of marks.

2. Validate every mark.

3. Calculate each student's total and average.

4. Find the index of the maximum total.

5. Print student-wise results and the top performer.

Flowchart

```
[Start] → [Input 3×3 marks matrix] → {All marks in 0..100?}\n No → [Invalid] → [End]\n Yes → [Calculate each
```

Pseudocode

```
BEGIN\n INPUT marks as 3x3 nested list\n IF any mark outside 0..100: PRINT invalid\n ELSE:\n   totals = []\n   FOR each row i
```

Python Source Code

```
marks = []
for student in range(3):
    row = []
    for subject in range(3):
        mark = int(input(f"Student {student + 1}, subject {subject + 1}: "))
        row.append(mark)
    marks.append(row)

if any(mark < 0 or mark > 100 for row in marks for mark in row):
    print("Invalid marks: each mark must be between 0 and 100")
else:
    totals = []
    averages = []
    for row in marks:
        total = sum(row)
        totals.append(total)
        averages.append(total / len(row))

    for i in range(3):
        print(f"Student {i + 1}: total={totals[i]}, average={averages[i]:.2f}")
    top_index = totals.index(max(totals))
    print(f"Top performer: Student {top_index + 1} ({totals[top_index]} marks)")
```

Test Cases

Matrix rows	Expected result
80, 70, 90; 60, 75, 65; 90, 90, 85	Totals 240, 200, 265; averages 80.00, 66.67, 88.33; Student 3 top
50, 50, 50; 50, 50, 50; 40, 40, 40	Totals 150, 150, 120; Student 1 top by tie rule
One mark = 101	Invalid marks

Sample Output

```
Student 1: total=240, average=80.00
Student 2: total=200, average=66.67
Student 3: total=265, average=88.33
```

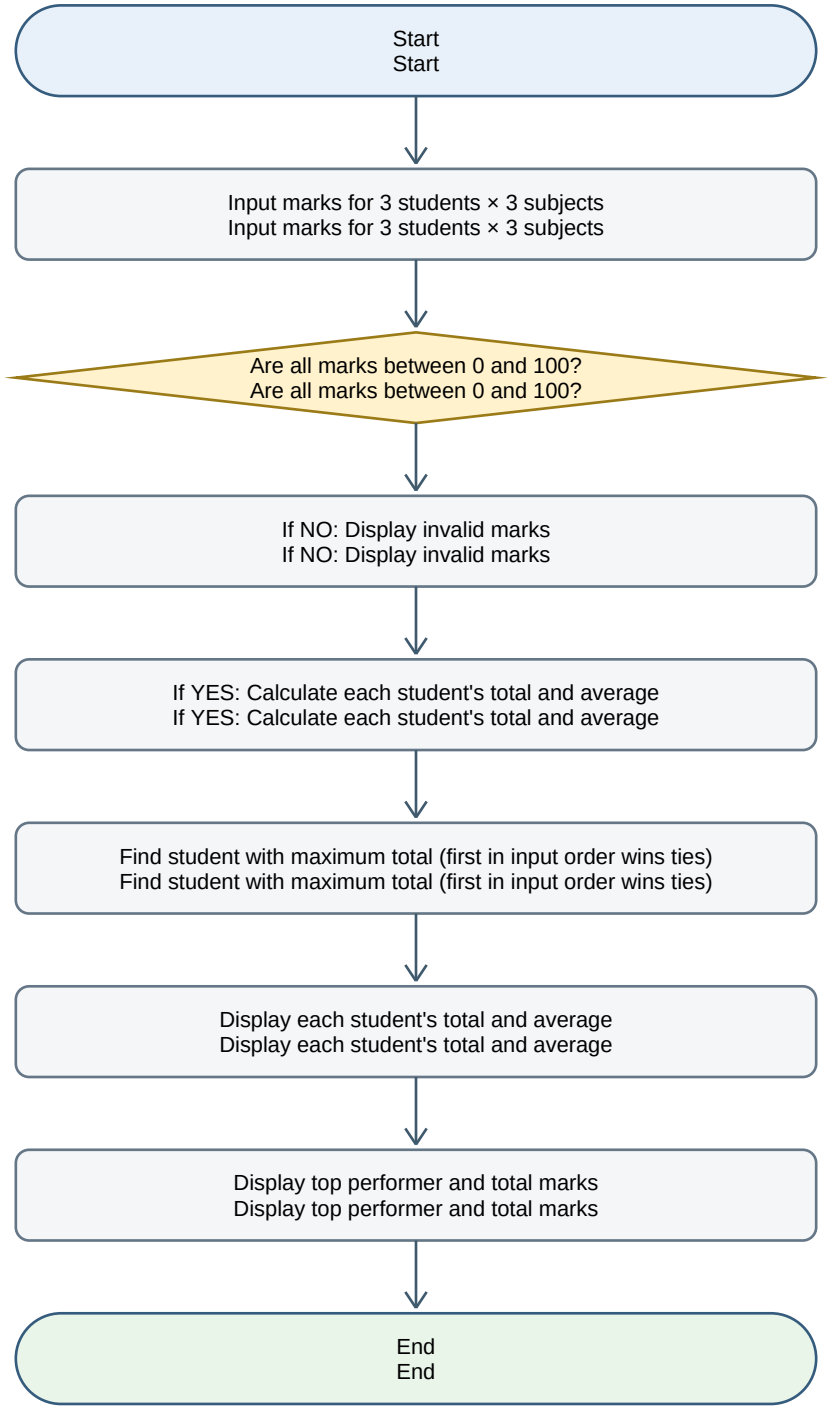
```
Top performer: Student 3 (265 marks)
```

Conclusion

Nested lists provide a simple representation for tabular marks, and row totals support student comparisons.

9. Student Marks Matrix Analysis

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.



10. Password Policy Validation Engine

Problem Statement

Validate sample passwords against minimum length, uppercase, lowercase, digit, and special-character requirements.

Objectives

Use string methods, loops, Boolean conditions, and clear validation messages.

Assumptions

A valid password must contain at least 8 characters, one uppercase letter, one lowercase letter, one digit, and one character from the set @#$%&*!?.

Algorithm

1. Read a password string.

2. Check its length and required character categories.

3. Collect every failed condition in a list.

4. Print each failure or confirm that the password is valid.

Flowchart

```
[Start] → [Input password] → [Check length and character types]\n → [Collect failed rules] → {Any failures?}\r
```

Pseudocode

```
BEGIN\n INPUT password\n failures = empty list\n IF length < 8: add length error\n IF no uppercase: add uppercase error\n IF
```

Python Source Code

```
password = input("Enter password: ")
specials = "@#$%&*!?"
errors = []

if len(password) < 8:
    errors.append("At least 8 characters are required")
if not any(ch.isupper() for ch in password):
    errors.append("Include an uppercase letter")
if not any(ch.islower() for ch in password):
    errors.append("Include a lowercase letter")
if not any(ch.isdigit() for ch in password):
    errors.append("Include a digit")
if not any(ch in specials for ch in password):
    errors.append("Include a special character from @#$%&*!?")

if errors:
    print("Invalid password:")
    for error in errors:
        print("-", error)
else:
    print("Password is valid")
```

Test Cases

Password	Expected result
Campus@123	Valid
abc	Invalid: length, uppercase, digit, special character
Campus123	Invalid: special character
CAMPUS@123	Invalid: lowercase letter

Sample Output

```
Enter password: Campus@123
Password is valid
```

Conclusion

The validator checks each rule independently and gives the user actionable feedback for every failed condition.

10. Password Policy Validation Engine

Follow arrows from top to bottom. Decision conditions are written inside the decision boxes.

